

MODULE 3: DISTRIBUTED DATABASES

UNIT 1: DISTRIBUTED DATABASE SYSTEM (DDB)

3.1 Introduction:

Database is primarily used for storing and manipulating the data for the organization or any particular requirement. It contains data and its related architectures. In ideal setting, any database will have its server and more than one user. Hence when database is designed, its server is kept at one place (or computer) and users are made to access this system. A distributed database is a type of database in which storage devices are not all attached to a common central processing unit. The data may be stored in multiple computers located in the same physical location, or may be dispersed over a network of interconnected computers. This unit gives an overview of distributed databases and their various forms

3.1.1 Objectives

By the end of this unit, you should be able to:

1. define Distributed database
2. know the features of distributed Databases
3. state the advantages and disadvantages of distributed databases

3.1.2 What is Distributed Database System?

A distributed database is a type of database in which storage devices are not all attached to a common central processing unit. The data may be stored in multiple computers located in the same physical location, or may be dispersed over a network of interconnected computers. Collections of data can be distributed across multiple physical locations. A distributed database can reside on network servers on the internet, corporate intranets or other company networks. The replication and distribution of databases improves database performance at end-user places.

Initially when database is created, it will be like a skeleton. As and when user starts accessing the database, its size grows or shrinks. Usually the size of the database grows

drastically than shrinking. Similarly number of users may also increase. These users may not be from one single location.

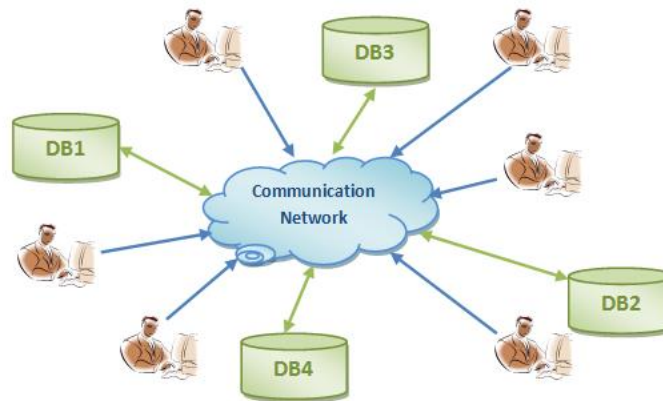


Figure 3.1: Structure of Distributed Database System

They will be from around the world. Hence the transaction with the database also increases. But this will create heavy network load as the users are at different location and server is at some other remote location. All these increasing factors affect the performance of database – it reduces its performance. But imagine systems like trading, bank accounts, etc which gives such a slow performance will lead to issues like concurrency, redundancy, security etc. Moreover users have to wait for longer time for their transaction to get executed. User cannot sit in front of their monitor to see the balance in their account for longer time.

In order to overcome these issues, a new way of allocating users and DB server is introduced. This new method is known as Distributed Database System. In this method, database server is kept at different remote locations. That means different database server is created and are placed at different locations rather than at single location. This in turn kept in sync with each other to maintain the consistency. The users accessing the DB will access any of these DB servers over the network as if they are accessing the DB from single location. They will be able to access the server without knowing its location. This in turn reduced the accessing time for the user. i.e.; when a user issues a

query, the system will fetch the server near to that user and access will be provided to nearest server. Hence it reduces the accessing time and network load too.

In ensuring that the distributive databases are up to date and current, there are two processes which are replication and duplication. Replication is using the specialized software that looks for changes in the distributive database. Once the changes have been identified, the replication process makes the entire database look the same. The replication process can be complex and also requires a lot of time with computer resources. This will depend on the size and number of the distributive database. Duplication on the other hand identifies one database as a master and then duplicates that database. A distributed database does not share main memory.

A database user accesses the distributed database through;

1. Local application which does not require data from other sites
2. Global applications which do require data from other sites.

3.1.3 Types of Distributed Database Systems

There are two types of DDB systems; Homogeneous and Heterogeneous

1. Homogeneous DDB

This type of distributed database system will have identical database systems distributed over the network. When we say identical database systems it includes software, hardware, operating systems etc – in short all the components that are essential for having a DB. For example a database system with Oracle alone distributed over the network, or with DB2 alone distributed over the network etc. this type of DDBMS system does not give the feel that they are located at different locations. Users access them as if they are accessing the same system.

Heterogeneous DDB

This is contrast to above concept. Here we will have different DBs distributed over the network. For example DB at one location can be oracle; at another location can be Sybase, DB2 or SQL server. In other words, in this type of DDB,

at least one of the DB is different from other DBs. In addition to this, the operating systems that they are using can also be different – one DB may be in Windows system while other would be in LINUX. Irrespective of the type of DDBMS used, user will be accessing these DBs as if they are accessing it locally.

3.1.4 Advantages and Disadvantages of Distributed Databases

Although DBs help in performance, security and recovery, there are many other advantages of this type of DBs.

1. **Transparency Levels:** In this system, physical location of the different DBs, the data like files, tables and any other data objects are not known to the users. They will have the illusion that they are accessing the single database at one location. Thus this method gives the distribution transparency about the databases. In addition, the records of the tables can also be distributed over the databases either wholly or partially by fragmenting them. This type of system provides location transparency by allowing the user to query any database or tables from any location, network transparency by allowing to access any DB over the network, naming transparency by accessing any names of objects like tables, views etc, replication transparency by allowing to keep the copies of the records at different DBs, fragmentation transparency by allowing to divide the records in a table horizontally or vertically.
1. **Availability and Reliability:** Distribution of data among different DBs allows the user to access the data without knowing failure of any one of the system. If any system fails or crashes, data will be provided from other system. For example, if DB-IN fails, the user will be given data from DB-ALL or vice versa. User will not be given message that the data is from DB-IN or DB-ALL, i.e.; all the DBs will be in sync with each other to survive the failure. Hence data will be available to the user all the time. This will in turn guarantee the reliability of the system.

2. **Performance :** Since the users access the data that are present in the DBs which are near to them, it reduces the network load and network time. This also reduces the data management time. Hence this type of systems gives high performance.
3. **Modularity :** Suppose any new DB has to be added to this system. This will not require any complex changes to the existing system. Any new DBs can be easily added to the system, without having much change to the existing system (because the entire configuration to have multiple DB already exists in the system). Similarly, if any DB has to be modified or removed, it can also be done without much effort.

3.1.5 Disadvantages

This system also has disadvantages.

1. **Increased Complexity:** This is the main drawback of this system. Since it has many DBs, it has to maintain all of them to work together. This needs extra design and work to keep them in sync, coordinate and make them work efficiently. These extra changes to the architecture makes DDBMS complex than a DB with single server.
2. **Very Expensive:** Since the complexity is increased, cost of maintaining these complexity also increases. Cost for multiple DBs and manage them are extra compared to single DB.
3. **Difficult to maintain Integrity:** Extra effort is needed to maintain the integrity among the DBs in the network. It may need extra network resources to make it possible.
4. **Security:** Since data is distributed over the DBs and network, extra caution is to be taken to have security of data. The access levels as well unauthorized access over the network needs some extra effort.
5. DDBMS requires very good experience in DBMS to deal with.
6. **Fragmentation** of data and their distribution gives extra challenges to the developer as well as database design. This in turn increases the complexity of database design to meet the transparency, reliability, integrity and redundancy.

1. Components of Distributed Database Systems

7. Distributed Database System consists of the various components:
8. **Database manager** is one of major component of Distributed Database systems. He/she is responsible for handling a segment of the distributed database.
9. **User Request Interface** is another important component of distributed database systems. It is usually a client program which acts as an interface to the Distributed Transaction Manager.
10. **Distributed Transaction Manager** is a program that helps in translating the user requests and converting into format required by the database manager, which are typically distributed. A distributed database system is made of both the distributed transaction manager and the database manager.

3.1.7 Current Trends in Distributed Databases

Current trends in distributed data management are centered on the Internet, in which petabytes of data can be managed in a scalable, dynamic, and reliable fashion. Two important areas in this direction are *cloud computing and peer-to-peer databases*.

Cloud Computing

Cloud computing is the paradigm of offering computer infrastructure, platforms, and software as services over the Internet. It offers significant economic advantages by limiting both up-front capital investments toward computer infrastructure as well as total cost of ownership. It has introduced a new challenge of managing petabytes of data in a scalable fashion. Traditional database systems for managing enterprise data proved to be inadequate in handling this challenge, which has resulted in a major architectural revision.

Peer-to-Peer Database Systems

A peer-to-peer database system (PDBS) aims to integrate advantages of P2P (peer-to-peer) computing, such as scalability, attack resilience, and self-

organization, with the features of decentralized data management. Nodes are autonomous and are linked only to a small number of peers individually. It is permissible for a node to behave purely as a collection of files without offering a complete set of traditional DBMS functionality. While FDBS and MDBS mandate the existence of mappings between local and global federated schemas, PDBSs attempt to avoid a global schema by providing mappings between pairs of information sources. In PDBS, each peer potentially models semantically related data in a manner different from other peers, and hence the task of constructing a central mediated schema can be very challenging. PDBSs aim to decentralize data sharing.

Problems in Distributed Database

One of the major problems in distributed systems is deadlock. A deadlock is a state where a set of processes request resources that are held by other processes in the set and none of the process can be completed. One process can request and acquire resources in any order without knowing the locks acquired by other processes. If the sequence of the allocations of resources to the processes is not controlled, deadlocks can occur. Hence we focus on deadlock detection and removal.

Deadlock Detection:

In order to detect deadlocks, in distributed systems, deadlock detection algorithm must be used. Each site maintains a local wait for graph. If there is any cycle in the graph, there is a deadlock in the system. Even though there is no cycle in the local wait for graph, there can be a deadlock. This is due to the global acquisition of resources.

In order to find the global deadlocks, global wait for graph is maintained. This is known as centralized approach for deadlock detection. The centralized approach to deadlock detection, while straightforward to implement, has two main drawbacks.

1. First, the global coordinator becomes a performance bottleneck, as well as a single point of failure.

2. Second, it is prone to detecting non-existing deadlocks, referred to as phantom deadlocks.

1. Deadlock Recovery

A deadlock always involves a cycle of alternating process and resource nodes in the resource graph. The general approach for deadlock recovery is process termination. In this method, nodes and edges of the resource graph are eliminated. In Process Termination, the simplest algorithm is to terminate all processes involved in the deadlock. This approach is unnecessarily wasteful, since, in most cases, eliminating a single process is sufficient to break the deadlock. Thus, it is better to terminate processes one at a time, release their resources, and check at each step if the deadlock still persists. Before termination of process following parameters need to be checked:

- a) The priority of the process:
- b) The cost of restarting the process
- c) The current state of the process

3.1.8 Conclusion

A distributed database is a type of database in which storage devices are not all attached to a common central processing unit. The data may be stored in multiple computers located in the same physical location, or may be dispersed over a network of interconnected computers.

3.1.9.0 Summary

In this unit we have learnt that:

- i. a distributed database is a type of database in which storage devices are not all attached to a common central processing unit. The data may be stored in multiple computers located in the same physical location, or may be dispersed over a network of interconnected computers.
- ii) The two types of DDB are Homogeneous and heterogeneous systems
- iii) Advantages and disadvantages of DDBMS

3.1.10 Tutor Marked Assignment

1. Define DDBs and state its advantages and disadvantages
2. Give suitable structure of a distributed database System
3. Enumerate the challenges of distributed database systems

1. Further Reading and other Resources

1. **David M. Kroenke, David J. Auer** (2008). Database Concepts. New Jersey . Prentice Hall
2. **Elmasri Navathe** (2003). Fundamentals of Database Systems. England. Addison Wesley.
3. **Fred R. McFadden, Jeffrey A. Hoffer** (1994). Modern Database management. England., Addison Wesley Longman